

Dokumen Analisis Risiko Keamanan Informasi dan Rencana Mitigasi

SmartStock Pro - PT Maju Bersama Digital

Sebagai sistem manajemen inventaris real-time yang memproses rantai pasok dan data keuangan PT Maju Bersama Digital, keamanan informasi pada website "SmartStock Pro" adalah prioritas utama. Dokumen ini menganalisis potensi ancaman keamanan informasi beserta langkah mitigasi teknis yang diimplementasikan.

1. Matriks Analisis Risiko Keamanan (OWASP Top 10 Context)

Berikut adalah identifikasi risiko keamanan utama yang relevan dengan aplikasi SmartStock Pro:

ID	Kategori Ancaman	Dampak	Probabilitas	Tingkat Risiko	Deskripsi Singkat
R1	SQL Injection (SQLi)	Kritis (High)	Rendah (Low)	Tinggi	Penyerang memanipulasi input form pencarian produk atau transaksi untuk membaca, memodifikasi, atau menghapus database secara ilegal.
R2	Cross-Site Scripting (XSS)	Sedang (Medium)	Sedang (Medium)	Sedang	Penyerang menyisipkan skrip JavaScript berbahaya ke nama produk atau deskripsi supplier yang kemudian dieksekusi di browser pengguna lain (misalnya

					pencurian session cookie).
R3	Cross-Site Request Forgery (CSRF)	Tinggi (High)	Sedang (Medium)	Tinggi	Penyerang menjebak pengguna yang sedang aktif login untuk mengirimkan permintaan tidak sah (seperti transfer barang siluman) tanpa disadari oleh pengguna.
R4	Brute-Force & Session Hijacking	Kritis (High)	Sedang (Medium)	Tinggi	Penyerang menebak password akun Manajer secara berulang atau mencuri ID session pengguna yang sedang aktif melalui jaringan tidak aman.
R5	Race Conditions (Stok Negatif)	Tinggi (High)	Tinggi (High)	Tinggi	Dua transfer stok paralel terjadi bersamaan untuk satu produk yang sama sehingga stok dihitung salah dan mengakibatkan stok bernilai negatif.

2. Langkah Mitigasi Teknis yang Diimplementasikan

A. Mitigasi Terhadap SQL Injection (R1)

- **Implementasi:** Penggunaan **Parameterized Queries** atau **Prepared Statements** menggunakan pustaka `sqlite3` bawaan Node.js.
- **Contoh Kode Teknis:**

```
// Kode Aman (Prepared Statement)
const stmt = db.prepare('SELECT * FROM products WHERE sku
stmt.all([userSkuInput], (err, rows) => { ... });
```

Dengan metode ini, input dari pengguna dianggap murni sebagai data literal dan tidak akan pernah dieksekusi sebagai perintah SQL aktif oleh parser database.

B. Mitigasi Terhadap Cross-Site Scripting / XSS (R2)

- **Implementasi:**

1. **Sanitasi Input:** Membersihkan tag HTML dari seluruh input teks pengguna menggunakan fungsi pembersih regex khusus sebelum data disimpan ke database.
2. **Kontekstual Output Encoding:** Saat menampilkan data dari database ke frontend (DOM), JavaScript wajib menggunakan properti `textContent` atau melakukan escape karakter khusus HTML (& menjadi `&`, < menjadi `<`, dsb.) daripada menggunakan `innerHTML`.
3. **Content Security Policy (CSP):** Menyertakan header HTTP CSP yang membatasi eksekusi inline script dan membatasi pemuatan file script hanya dari domain yang tepercaya.

C. Mitigasi Terhadap CSRF (R3)

- **Implementasi:**

1. **Token CSRF Dinamis:** Server membuat token CSRF unik yang disimpan di session pengguna saat pertama kali login.
2. **Validasi Request:** Untuk setiap request mutasi data (POST, PUT, DELETE), middleware keamanan backend akan memverifikasi keberadaan dan kecocokan token CSRF yang dikirimkan oleh klien dalam header HTTP X-CSRF-Token dengan token yang tersimpan di server-side session. Jika tidak cocok, server menolak akses dengan kode HTTP 403 Forbidden.

D. Mitigasi Terhadap Brute-Force & Session Hijacking (R4)

- **Implementasi:**

1. **Password Hashing Kuat:** Menggunakan `bcryptjs` dengan salt minimum 10 putaran untuk mengenkripsi password. Skema enkripsi ini tahan terhadap serangan kamus (*dictionary attacks*).
2. **Session Cookie Security:** Session diatur menggunakan properti:
 - `HttpOnly`: Mencegah script JavaScript klien (termasuk potensi serangan XSS) mengakses isi cookie session.

- **Secure:** Memastikan cookie hanya ditransmisikan melalui koneksi terenkripsi HTTPS (port 443).
- **SameSite=Strict:** Mencegah pengiriman cookie session dalam request lintas situs (proteksi CSRF tambahan).

3. **Automatic Timeout:** Jika pengguna tidak aktif selama 15 menit, session server otomatis dimatikan (*destroyed*) untuk mencegah penyalahgunaan komputer yang ditinggalkan staf di area gudang.

E. Mitigasi Terhadap Race Conditions & Data Lock (R5)

- **Implementasi:**

1. **Database Transaction Lock:** Setiap kali transaksi transfer stok atau transaksi barang masuk/keluar dijalankan, sistem menggunakan blok transaksi `BEGIN EXCLUSIVE TRANSACTION` di SQLite. Hal ini mengunci proses penulisan tabel lain hingga transaksi saat ini selesai di-COMMIT atau di-ROLLBACK.
2. **FIFO/LIFO Batch Allocation:** Logika bisnis mengharuskan pengecekan ketersediaan stok terkini secara real-time langsung dari dalam blok transaksi database terisolasi untuk memastikan data sisa kuantitas di batch (`remaining_qty`) akurat.